Лабораторная работа №3

Объектно-ориентированное программирование в Python

1. Цель работы

Освоить классы, наследование, инкапсуляцию, свойства, переопределение методов, полиморфизм и композицию в объектной модели спортивного клуба.

2. Постановка индивидуального задания

Вариант 12 — «Спортивный клуб». Разработать объектную модель учёта спортсменов.

Программа должна:

– Реализовать базовый класс Athlete и классы Runner и Swimmer.

– Для бегуна учитывать результат на дистанции, для пловца — длину бассейна и время.

– Обеспечить единый метод отображения результата, обновление достижения и класс SportClub.

3. Краткие теоретические сведения

ООП организует программу как взаимодействие объектов с состоянием и поведением. Athlete задаёт общие свойства спортсмена, а Runner и Swimmer наследуют их и переопределяют result().

Инкапсуляция результата реализована через свойство @property и setter, который запрещает неположительное время. Метод train() обновляет лучший результат только в случае улучшения.

Полиморфизм позволяет вызвать result() у всех спортсменов из списка SportClub, не проверяя их конкретные типы. Отношение «клуб содержит спортсменов» демонстрирует композицию объектов.

4. Исходный код программы

```
class Athlete:
    def __init__(self, name, best_time):
        self.name = name
        self.best_time = best_time

    @property
    def best_time(self):
        return self._best_time

    @best_time.setter
    def best_time(self, value):
        if value <= 0:
            raise ValueError("Время должно быть положительным")
        self._best_time = value

    def train(self, time):
        if time <= 0:
            raise ValueError("Время должно быть положительным")
        self.best_time = min(self.best_time, time)

    def result(self):
        return f"{self.name}: {self.best_time} с"


class Runner(Athlete):
    def __init__(self, name, best_time, distance):
        super().__init__(name, best_time)
        self.distance = distance

    def result(self):
        return f"Бегун {self.name}: {self.distance} м за {self.best_time} с"


class Swimmer(Athlete):
    def __init__(self, name, best_time, pool_length):
        super().__init__(name, best_time)
        self.pool_length = pool_length

    def result(self):
        return f"Пловец {self.name}: бассейн {self.pool_length} м за {self.best_time} с"


class SportClub:
    def __init__(self, name):
        self.name = name
        self.athletes = []

    def add(self, athlete):
        self.athletes.append(athlete)

    def show(self):
        print("Клуб:", self.name)
        for athlete in self.athletes:
            print(athlete.result())
```

```
def main():
    club = SportClub("Старт")
    runner = Runner("Максим", 14.0, 100)
    club.add(runner)
    club.add(Runner("Иван", 28.0, 200))
    club.add(Swimmer("Анна", 35.0, 50))
    club.show()
    runner.train(13.5)
    print("После тренировки:", runner.result())


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 3.1 показано отображение спортсменов, а также обновление результата бегуна после тренировки.

```
python main.py
Клуб: Старт
Бегун Максим: 100 м за 14.0 с
Бегун Иван: 200 м за 28.0 с
Пловец Анна: бассейн 50 м за 35.0 с
После тренировки: Бегун Максим: 100 м за 13.5 с
```

Рисунок 3.1 – Работа объектной модели спортивного клуба

6. Вывод

Реализована иерархия Athlete, Runner и Swimmer и объект SportClub. Показаны свойства, валидация рекорда, наследование, переопределение методов и полиморфный вывод спортсменов.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	